Multi-Precision Arithmetic Calculator (Large Numbers)

Computer Organization and Assembly Language Lab Project

Group Members and Modules

Member 1: Aleeza Noor - Input, validation, and storage module
Member 2: Hina Naeem - Addition, subtraction, and comparison module
Member 3: Ayesha Khan - Multiplication module
Member 4: Urwa Abbasi - Main menu, output, analysis, and integration module

1. Introduction

The Multi-Precision Arithmetic Calculator is an Assembly Language console application designed to perform arithmetic operations on numbers larger than the normal 32-bit register limit. The project implements large-number addition, subtraction, and multiplication using arrays of digits. It follows a modular development structure so that each member contributes a separate technical component and also participates in testing and integration.

2. Problem Statement

Standard integer registers cannot directly store very large numbers such as 50-digit or 100-digit values. The problem is to design a calculator that accepts large numeric strings from the user, validates the input, stores the digits in memory, performs arithmetic manually digit by digit, and displays the correct result. The program must also handle negative numbers and invalid input.

3. Objectives

The main objectives are:
- Read large numbers from user input.
- Validate numeric input and reject invalid characters.
- Store numbers as reversed digit arrays.
- Support positive and negative signs.
- Perform large-number addition, subtraction, and multiplication.
- Display results clearly through a menu-driven console interface.
- Provide analysis related to digit counts, memory usage, and system behavior.
- Use GitHub collaboration with visible member contributions.

4. Tools and Libraries

The project was developed using Microsoft Visual Studio 2022, MASM, and the Irvine32 library. GitHub and GitHub Desktop were used for version control and collaboration.

5. System Design

The program is divided into multiple Assembly source files. The main module controls the menu and user flow. The input module reads user input, validation checks correctness, and storage converts valid strings into reversed digit arrays. Arithmetic modules process the arrays and store results in a shared result array. The output module prints the final result, while the analysis module displays digit counts and fixed memory usage.

Data Flow:

User input -> ReadNumber -> ValidateNumber -> StringToArray -> Arithmetic module -> result array -> DisplayResult -> RunAnalysis

6. Module Distribution and Team Contribution

Member 1 led the input and storage module. This included reading large numbers, checking invalid characters, handling signs, and converting input strings into reversed digit arrays. Member 1 also helped test input-output behavior during integration.

Member 2 led the arithmetic core for addition and subtraction. This included comparing large numbers, handling carry and borrow, and determining result signs. Member 2 also helped ensure arithmetic modules followed the shared storage format.

Member 3 led multiplication. The multiplication module uses long multiplication on reversed arrays and stores the product in the shared result array. It handles carry propagation, negative signs, and zero inputs.

Member 4 led the main menu, output formatting, analysis display, and final integration. This module connects all procedures and provides the complete user workflow.

7. Implementation Details

The project stores each large number in a byte array where each byte contains one digit from 0 to 9. Digits are stored in reverse order so that index 0 contains the least significant digit. This makes arithmetic easier because addition, subtraction, and multiplication all begin from the rightmost digit.

The sign of each number is stored separately. A value of 0 means positive and a value of 1 means negative. This avoids mixing sign characters with digit arrays. The result uses the same format: result array, result length, and result sign.

Important Shared Variables:

```
num1, num2       - reversed digit arrays
result           - reversed result array
num1Len, num2Len, resultLen - digit counts
num1Sign, num2Sign, resultSign - sign flags
```

8. Addition and Subtraction Logic

Addition works digit by digit with carry. If both numbers have the same sign, their magnitudes are added and the result keeps the same sign. If the signs are different, the program compares magnitudes and subtracts the smaller magnitude from the larger one.

Subtraction is implemented by flipping the sign of the second number and reusing the addition logic. This keeps the code simpler and reduces repeated sign-handling logic.

9. Multiplication Logic

Multiplication uses the standard long multiplication method. Every digit of the first number is multiplied by every digit of the second number. The partial products are added at index i + j because the arrays are reversed. Carry is propagated after each digit multiplication. The final result is trimmed to remove unnecessary leading zeroes.

10. Integration Strategy

Each module was first tested independently using temporary test main files. These temporary files were not kept as final project source files. After module testing, the final main menu was connected with all shared procedures. The integrated project was then rebuilt and tested through Visual Studio.

Only one final file contains main PROC: main.asm. Other files expose procedures using PUBLIC and access shared variables using EXTERN. This avoids duplicate entry points and linker errors.

11. Testing and Debugging Strategy

The project was tested using valid large numbers, negative numbers, zero cases, carry-heavy cases, and invalid input. Linker errors during development were resolved by matching procedure declarations and using PROTO/PUBLIC/EXTERN consistently. Duplicate test main files were excluded from the final build.

Final Test Cases:

Addition: 999999 + 888888888888 = 888889888887
Subtraction: 10000000000000000000 - 9999999999999999999 = 1
Negative subtraction: 123456789 - 987654321 = -864197532
Multiplication: 12345678901234567890 * 9876543210 = 121932631124828532111263526900
Negative multiplication: 9999999000 * -333 = -3329999667000
Zero multiplication: 0 * 98765432109876543210 = 0
Invalid input: 999999999aaa is rejected.

12. Analysis and Evaluation

The maximum input size is 200 digits per number. The first number uses 200 bytes, the second number uses 200 bytes, and the result array uses 400 bytes. Addition and subtraction have O(n) time complexity. Multiplication has O(n*m) time complexity because it compares each digit of one input with each digit of the other input.

The program is efficient for the project scope because all storage is fixed and predictable. The reversed array format reduces complexity in arithmetic operations.

13. GitHub Collaboration Workflow

The group used GitHub for collaboration. Members committed their own module files and screenshots. Final integration was tested after all modules were available. During integration, some placeholder files and duplicate copies were cleaned so that the final Code folder contains

one clear set of source files.

14. Challenges Faced

The main challenges were managing procedure names between modules, avoiding duplicate main PROC definitions, handling GitHub account/commit identity correctly, and keeping Visual Studio build settings consistent. Another challenge was ensuring old placeholder files did not overwrite final module code during integration. These issues were resolved through careful rebuild testing and cleanup commits.

15. Conclusion

The final project successfully implements a modular multi-precision arithmetic calculator in Assembly Language. It supports large-number input, validation, reversed-array storage, addition, subtraction, multiplication, output formatting, and basic analysis. The project follows the required modular development strategy and demonstrates collaborative implementation, testing, debugging, and integration.